Cake Pattern

Cake · Wire UserRepo+Email into UserService app · Build with Waqar

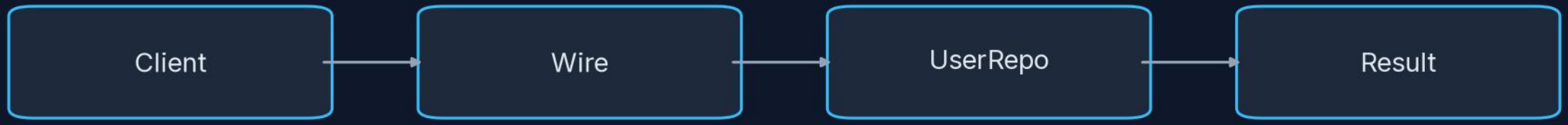
Today's Hook

- Problem: Wire UserRepo+Email into UserService app
- Pattern: Cake — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Wire UserRepo+Email into UserService app
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Cake: Wire UserRepo+Email into UserService app

Real-World Scenario

- Running example: Wire UserRepo+Email into UserService app
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.cake;

import com.buildwithwaqar.patterns.functional.cake.CakePattern;
/** Problem: assemble UserRepo + EmailService into UserService without a DI framework. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Cake problem: wire user welcome flow ===");
    System.out.println(new CakePattern.App().userService().welcome("42"));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.cake;

/**
 * Cake pattern (Scala) emulated with Java interfaces + default methods.
 * Self-type: a component declares what it needs via abstract getters;
 * the "cake" assembles mixins into one object.
 */
public final class CakePattern {
    private CakePattern() {}

    public interface UserRepoComponent {
        UserRepo userRepo();
    }

    public interface UserRepo {
        String findEmail(String id);
    }

    public interface EmailServiceComponent {
        EmailService emailService();
    }
```

Solution (2)

```
public interface EmailService {
    String send(String to, String body);
}

public interface UserServiceComponent extends UserRepoComponent, EmailServiceComponent {
    default UserService userService() {
        return id -> {
            var email = userRepo().findEmail(id);
            return emailService().send(email, "welcome " + id);
        };
    }
}

public interface UserService {
    String welcome(String id);
}

/** The cake — wires live implementations. */
public static final class App implements UserServiceComponent {
    public UserRepo userRepo() { return id -> id + "@example.com"; }
    public EmailService emailService() { return (to, body) -> "sent:" + to + ":" + body; }
}
```

Solution (3)

```
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.cake;

import com.buildwithwaqar.patterns.functional.cake.CakePattern;
/** Problem: assemble UserRepo + EmailService into UserService without a DI framework. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Cake problem: wire user welcome flow ===");
    System.out.println(new CakePattern.App().userService().welcome("42"));
  }
}
```

Walkthrough Output

```
=== Cake problem: wire user welcome flow ===
sent:42@example.com:welcome 42
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Cake.

Related Patterns

- Contrast Cake with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Cake → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.